Jobsheet 9 – Abstract Classes and Interfaces

I. Competencies

After completing this worksheet, students are expected to be able to:

1. Explain the purpose of using *abstract classes* and *interfaces*;
2. Apply *abstract classes* in a program;
3. Apply *interfaces* and *multiple interfaces* in a program;
4. Distinguish when to use an *abstract class* and when to use an *interface*.

Time Allocation

This jobsheet is designed to be completed in **200 minutes**. Use the following time allocation as a guide.

Activity	Time
Reading the introduction and preparing the project	10 minutes
Experiment 1: Abstract Class (including questions)	40 minutes
Experiment 2: Interface (including questions)	45 minutes
Experiment 3: Multiple Interfaces (including questions)	40 minutes
Assignment	65 minutes
Total	**200 minutes**

II. Introduction

A. Abstract Class

An *abstract class* is a class that **cannot be instantiated** (no objects can be created from it with `new`), but it can be *extended*. An *abstract class* can only be used once it is *extended* by another class.

Characteristics:

a. It can have attributes, *constructors*, and regular *methods*, just like an ordinary class.

b. It can have *methods* without a body (declaration only), called *abstract methods*. A class that has at least one *abstract method* **must** be declared as an *abstract class*.

c. It is declared with the keywords `abstract class`.

d. Every subclass (that is not itself abstract) **must** *override* all *abstract methods* of its parent.

Purpose: to describe something general that can only work after it is described in a more specific form. Example: every animal can definitely move, but how it moves only becomes clear once we know which animal it is.

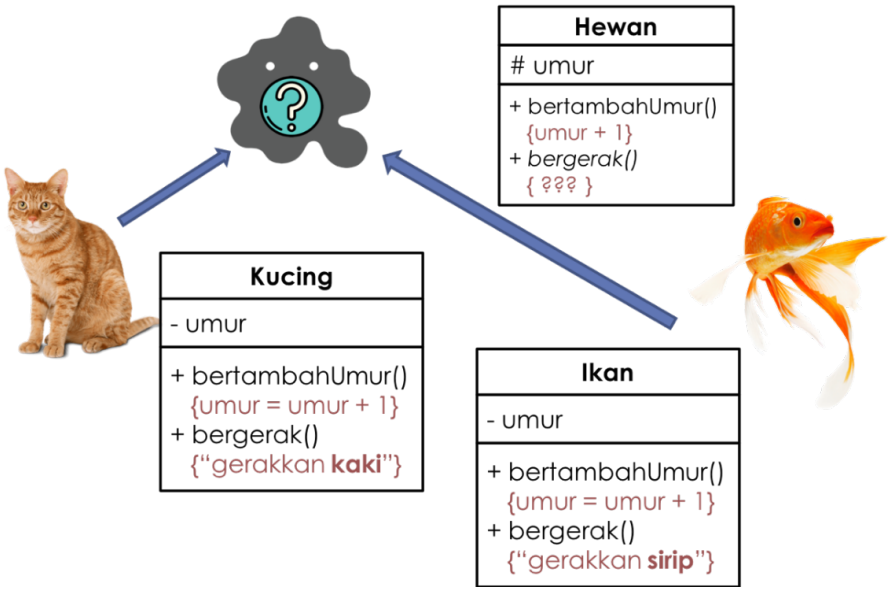


Figure 1. Illustration of the abstract class Hewan (Animal)

B. Interface

An *interface* is a **contract** containing a list of *methods* that a class **must** provide so that the class can be regarded as "something else". Example: a Sarjana (undergraduate) can be called *Cumlaude* (cum laude) if they fulfil the `ICumlaude` contract, that is, they can `lulus()` (graduate) and `meraihIPKTinggi()` (achieve a high GPA).

Characteristics:

a. It is declared with the keyword `interface` and used by a class with the keyword `implements`.

b. *Methods* in an interface are automatically `public abstract` (these modifiers may be omitted). Since Java 8, an interface may also have `default` and `static` methods with a body, but in this jobsheet we focus on *abstract methods*.

c. An interface cannot have regular variables; it may only have **constants** (automatically `public static final`).

d. An interface has no *constructor* and **cannot be instantiated** with `new`.

e. In this jobsheet, interface names start with a capital 'I' (e.g. `ICumlaude`) so they are easy to tell apart from classes. This is a naming habit, not a Java rule; the Java library itself uses names such as `Comparable` and `Runnable`.

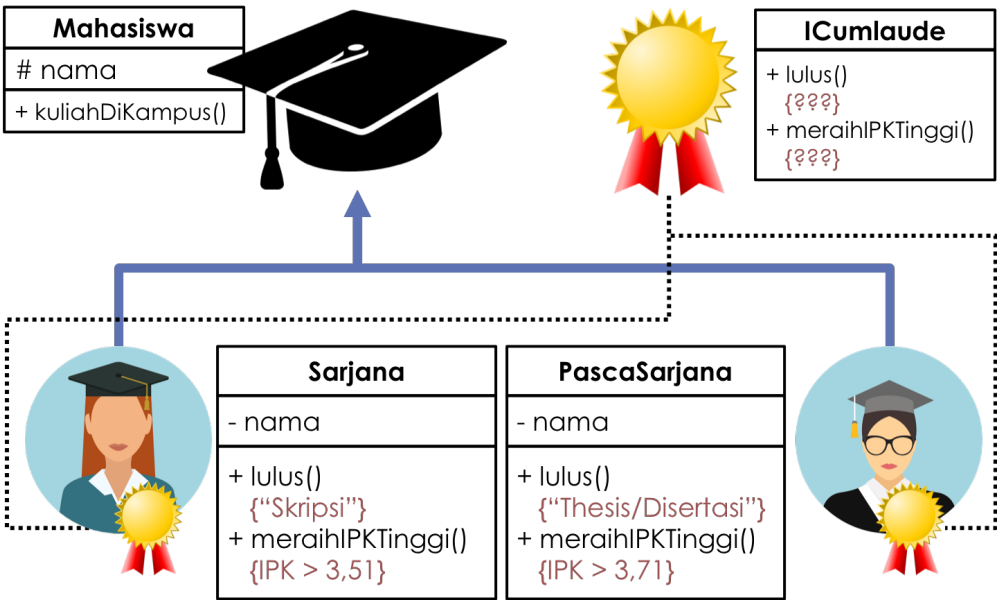


Figure 2. Illustration of the ICumlaude interface

Multiple interfaces. A class can *extend* only **one** class, but it can *implement* **more than one** interface (separated by commas), for example `class PascaSarjana extends Mahasiswa implements ICumlaude, IBerprestasi`. This is how Java gives a class several "capabilities" at once, as a substitute for *multiple inheritance*.

Rules that often cause errors:

a. A *method* that implements an interface method in a class **must** be `public`.

b. The name, number, and types of the *method* parameters in the class must exactly match those in the interface.

c. *Methods* and constants in an interface must not use the `private` or `protected` modifiers.

d. Interfaces and *abstract classes* cannot be instantiated with `new`; objects are created from the classes that *implement*/*extend* them.

C. UML Notation

- *Abstract class*: the class name and *abstract methods* are written in **italics** (you may also add «abstract»).
- *Interface*: the «interface» stereotype is written above the name; the interface name is **not** in italics.
- *Extends* (inheritance): a solid line with a hollow triangle arrowhead, pointing to the parent class.
- *Implements*: a **dashed** line with a hollow triangle arrowhead, pointing to the interface.

D. Differences Between Abstract Classes and Interfaces

Abstract Class	Interface
Can contain *abstract methods* and regular *methods* (with a body)	Contains *abstract methods* (since Java 8 also `default` and `static` methods)
Method modifiers are written explicitly (`public`, `protected`, etc.)	*Methods* are automatically `public abstract`; no need to write it

Abstract Class	Interface
Can have regular attributes (*instance variables*) and constants	Can only have constants (automatically `public static final`)
Can have a *constructor*	Has no *constructor*
A class can *extend* only **one** class (including an abstract class)	A class can *implement* **more than one** interface
An *abstract class* can *implement* one or more interfaces	An interface can *extend* other interfaces, but cannot *implement* anything
An "**is-a**" relationship (Kucing is a Hewan)	A "**can-do**" relationship (a Sarjana can become Cumlaude)

III. Practicum

Project Preparation

Create a new project in NetBeans named **PBOMinggu9**. In this project, create **two packages**: `abstractclass` for Experiment 1 and `interfacelatihan` for Experiments 2 and 3 (right-click *Source Packages* → **New** → **Java Package…**). Create each class/interface by right-clicking the package → **New** → **Java Class…** or **New** → **Java Interface…**.

Tip: after you type `extends` or `implements`, NetBeans shows a light-bulb warning icon. Click the icon and choose **Implement all abstract methods** to generate the *method* skeletons automatically, then replace `throw new UnsupportedOperationException(...)` with the code in this jobsheet.

Note on names: class and *method* names in the code (for example `Hewan` = animal, `Kucing` = cat, `Ikan` = fish, `Orang` = person, `bergerak()` = move, `Mahasiswa` = student, `Sarjana` = undergraduate, `PascaSarjana` = postgraduate, `Rektor` = rector) are kept as they are so that they match the class diagrams.

Experiment 1: Abstract Class (± 40 minutes)

All animals share some characteristics; for example, every animal has an age, and whatever the animal, its age increases by one every year. However, each animal **moves** differently: a cat moves by stepping with its legs, while a fish moves by moving its fins. Every pet owner can take their pet for a walk, but each pet moves in a different way. We will model this scenario using an **abstract class**.

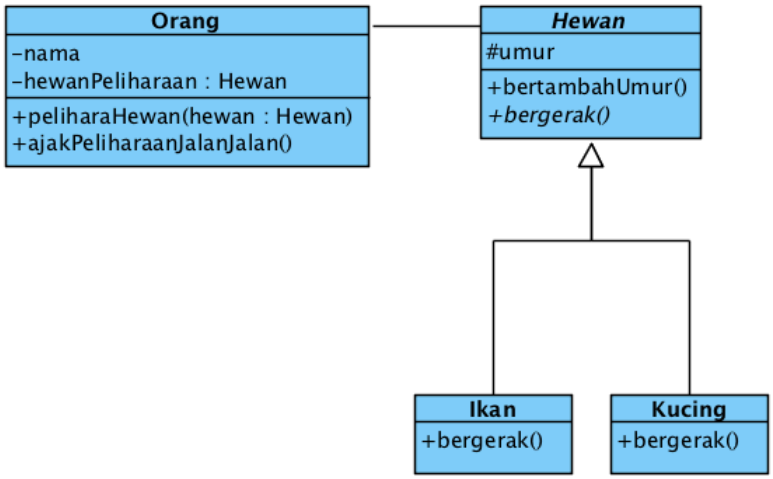


Figure 3. Class diagram for Experiment 1

1. In the `abstractclass` package, create the class **Hewan** as the following *abstract class*.

```
package abstractclass;

public abstract class Hewan {
    protected int umur;

    protected Hewan() {
        this.umur = 0;
    }

    public void bertambahUmur() {
        this.umur += 1;
    }

    public abstract void bergerak();
}
```

The class `Hewan` contains a regular attribute and *method*, plus the *abstract method* `bergerak()`, which has no body. Every subclass of `Hewan` will have to override this *method*.

2. Create the class **Kucing** that *extends* `Hewan`. Use the light-bulb icon → **Implement all abstract methods**, then change the body of the *method* `bergerak()` as follows.

```
package abstractclass;

public class Kucing extends Hewan {
    @Override
    public void bergerak() {
        System.out.println("Walking on LEGS, \"tap..tap..\"");
    }
}
```

3. In the same way, create the class **Ikan**.

```
package abstractclass;

public class Ikan extends Hewan {
    @Override
    public void bergerak() {
        System.out.println("Swimming with FINS, \"swish..swish..\"");
    }
}
```

4. Create a regular class **Orang**. This class is a user of the *abstract class* `Hewan`.

```
package abstractclass;

public class Orang {
    private String nama;
    private Hewan hewanPeliharaan;

    public Orang(String nama) {
        this.nama = nama;
    }

    public void peliharaHewan(Hewan hewan) {
        this.hewanPeliharaan = hewan;
    }

    public void ajakPeliharaanJalanJalan() {
        System.out.println("My name is " + this.nama);
        System.out.println("My pet moves by: ");
        this.hewanPeliharaan.bergerak();
        System.out.println("----------------------------------------");
    }
}
```

5. Create the *main class* **Program** in the same package.

```
package abstractclass;

public class Program {
    public static void main(String[] args) {
        Kucing kucingKampung = new Kucing();
        Ikan lumbaLumba = new Ikan();

        Orang ani = new Orang("Ani");
        Orang budi = new Orang("Budi");

        ani.peliharaHewan(kucingKampung);
        budi.peliharaHewan(lumbaLumba);

        ani.ajakPeliharaanJalanJalan();
        budi.ajakPeliharaanJalanJalan();
    }
}
```

6. Run the class `Program` (right-click → **Run File** or Shift + F6), then observe the result.

Output:

```
My name is Ani
My pet moves by:
Walking on LEGS, "tap..tap.."
----------------------------------------
My name is Budi
My pet moves by:
Swimming with FINS, "swish..swish.."
----------------------------------------
```

Experiment 1 Questions

After each test, restore the code to its original state before moving on to the next question.

1. Explain how the program above runs. Why does the call `this.hewanPeliharaan.bergerak()` in the class `Orang` produce different output for Ani and Budi?

2. Add the line `Hewan hewan = new Hewan();` to the `main()` method. Show the compilation result and explain it briefly.
3. Is a class that *extends* an *abstract class* allowed not to implement the *abstract methods* of its parent? Prove it by removing the *method* `bergerak()` from the class `Ikan`, then show the compilation result and explain it.
4. Add the *abstract method* `public abstract void bernapas();` (breathe) inside the class `Ikan` without changing the declaration of the class `Ikan`. Show the compilation result and explain which rule is violated.

Experiment 2: Interface (± 45 minutes)

At a graduation ceremony, the Rector will award a *Cum Laude* certificate to every student who meets the requirements. A **Sarjana** (undergraduate) student must complete an **undergraduate thesis** and have a GPA above 3.51, while a **Pasca Sarjana** (postgraduate) student must complete a **master's thesis** and have a GPA above 3.71. These requirements are expressed in the interface `ICumlaude`.

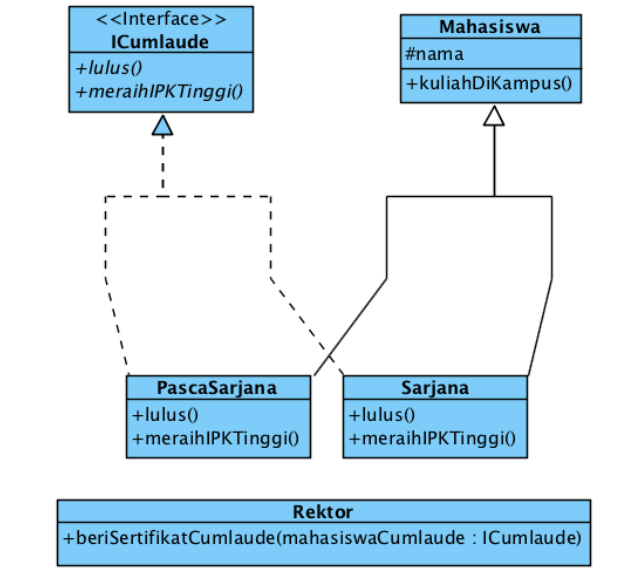


Figure 4. Class diagram for Experiment 2

1. In the `interfacelatihan` package, create the interface **ICumlaude** (right-click → **New** → **Java Interface…**) with two *abstract methods*: `lulus()` and `meraihIPKTinggi()`.

```
package interfacelatihan;

public interface ICumlaude {
    public abstract void lulus();
    public abstract void meraihIPKTinggi();
}
```

2. Create the class **Mahasiswa**.

```
package interfacelatihan;

public class Mahasiswa {
    protected String nama;

    public Mahasiswa(String nama) {
        this.nama = nama;
    }

    public void kuliahDiKampus() {
        System.out.println("I am a student, my name is " + this.nama);
        System.out.println("I attend classes on campus.");
    }
}
```

3. Create the class **Sarjana** that *extends* `Mahasiswa` and *implements* `ICumlaude`. Use the light-bulb icon to generate the *method* skeletons, then adjust their bodies.

```
package interfacelatihan;

public class Sarjana extends Mahasiswa implements ICumlaude {
    public Sarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("I have completed my UNDERGRADUATE THESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("My GPA is above 3.51");
    }
}
```

`Sarjana` *extends* `Mahasiswa`, which means **a Sarjana is a Mahasiswa**. For `Sarjana` objects to be called **Cumlaude**, the class must *implement* the interface `ICumlaude`.

4. In the same way, create the class **PascaSarjana**.

```
package interfacelatihan;

public class PascaSarjana extends Mahasiswa implements ICumlaude {
    public PascaSarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("I have completed my MASTER'S THESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("My GPA is above 3.71");
    }
}
```

5. Create the class **Rektor**. Notice that the parameter of the *method* `beriSertifikatCumlaude()` (award cum laude certificate) has the interface type `ICumlaude`.

```
package interfacelatihan;

public class Rektor {
    public void beriSertifikatCumlaude(ICumlaude mahasiswaCumlaude) {
        System.out.println("I am the RECTOR, granting a cum laude award.");
        System.out.println("Congratulations! Please introduce yourself..");

        mahasiswaCumlaude.lulus();
        mahasiswaCumlaude.meraihIPKTinggi();

        System.out.println("----------------------------------------");
    }
}
```

6. Create the *main class* **Program** in the `interfacelatihan` package.

```
package interfacelatihan;

public class Program {
    public static void main(String[] args) {
        Rektor pakRektor = new Rektor();

        Mahasiswa mhsBiasa = new Mahasiswa("Charlie");
        Sarjana sarjanaCumlaude = new Sarjana("Dini");
        PascaSarjana masterCumlaude = new PascaSarjana("Elok");

        pakRektor.beriSertifikatCumlaude(mhsBiasa);
        pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
        pakRektor.beriSertifikatCumlaude(masterCumlaude);
    }
}
```

7. If all classes are typed correctly, an **error** will appear and the program cannot run. Fix your code so that the program produces the following output.

Output:

```
I am the RECTOR, granting a cum laude award.
Congratulations! Please introduce yourself..
I have completed my UNDERGRADUATE THESIS
My GPA is above 3.51
----------------------------------------
I am the RECTOR, granting a cum laude award.
Congratulations! Please introduce yourself..
I have completed my MASTER'S THESIS
My GPA is above 3.71
----------------------------------------
```

Experiment 2 Questions

1. Why does an error occur in step 7? Write down the error message and explain how you fixed it.
2. Can the *method* `kuliahDiKampus()` be called from the object `sarjanaCumlaude` in the class `Program`? Why?
3. Can the *method* `kuliahDiKampus()` be called from the parameter `mahasiswaCumlaude` in the *method* `beriSertifikatCumlaude()` of the class `Rektor`? Why?
4. Remove the keyword `public` from the *method* `lulus()` in the class `Sarjana`. Show the compilation result and explain it, then restore the code.

Experiment 3: Multiple Interfaces (± 40 minutes)

At the same graduation ceremony, the Rector will also present the **Mawapres** award (*Mahasiswa Berprestasi*, Outstanding Student). The criteria for an outstanding student are **winning a competition** and **publishing a scientific article**. These criteria are expressed in a new interface, `IBerprestasi`, so `PascaSarjana` will *implement* two interfaces at once. This experiment continues the code from Experiment 2 in the same package.

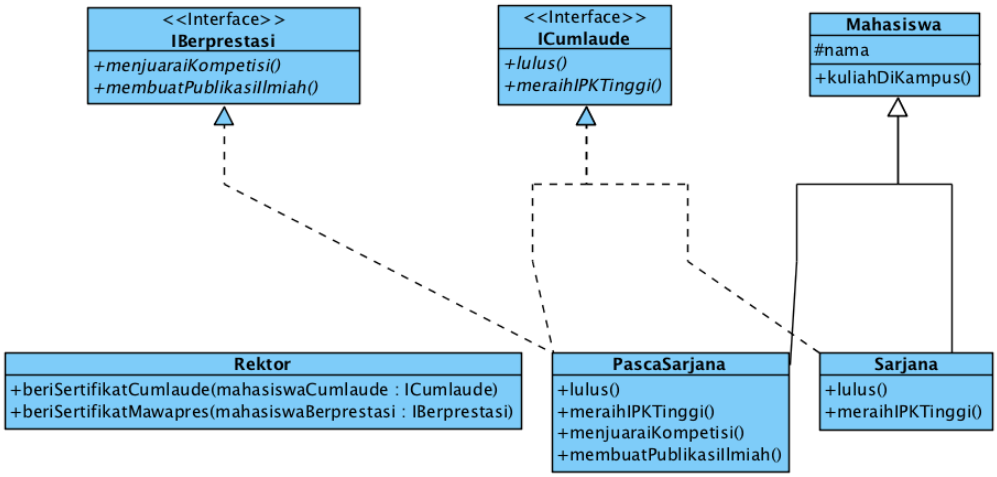


Figure 5. Class diagram for Experiment 3

1. In the `interfacelatihan` package, create a new interface **IBerprestasi**.

```
package interfacelatihan;

public interface IBerprestasi {
    public abstract void menjuaraiKompetisi();
    public abstract void membuatPublikasiIlmiah();
}
```

2. Change the declaration of the class **PascaSarjana** by adding `IBerprestasi` after the keyword `implements` (separated by a comma). Use the light-bulb icon to generate the *method* skeletons, then fill in both *methods* as follows.

```
public class PascaSarjana extends Mahasiswa
        implements ICumlaude, IBerprestasi {
    // constructor, lulus(), and meraihIPKTinggi() stay as before

    @Override
    public void menjuaraiKompetisi() {
        System.out.println("I have won an INTERNATIONAL competition");
    }

    @Override
    public void membuatPublikasiIlmiah() {
        System.out.println("I have published in an INTERNATIONAL journal");
    }
}
```

3. Add the *method* `beriSertifikatMawapres()` to the class **Rektor**.

```
public void beriSertifikatMawapres(IBerprestasi mahasiswaBerprestasi) {
    System.out.println("I am the RECTOR, granting a MAWAPRES award.");
    System.out.println("Congratulations! How did you achieve this?");

    mahasiswaBerprestasi.menjuaraiKompetisi();
    mahasiswaBerprestasi.membuatPublikasiIlmiah();

    System.out.println("----------------------------------------");
}
```

4. Modify the `main()` method in the class **Program**: comment out every line that calls `beriSertifikatCumlaude()`, then add calls to `beriSertifikatMawapres()` as follows.

```
public static void main(String[] args) {
    Rektor pakRektor = new Rektor();

    Sarjana sarjanaCumlaude = new Sarjana("Dini");
    PascaSarjana masterCumlaude = new PascaSarjana("Elok");

    // pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
    // pakRektor.beriSertifikatCumlaude(masterCumlaude);

    pakRektor.beriSertifikatMawapres(sarjanaCumlaude);
    pakRektor.beriSertifikatMawapres(masterCumlaude);
}
```

5. There will be an **error** in step 4, so the program cannot run. Fix the program code so that the output is the same as below.

Output:

```
I am the RECTOR, granting a MAWAPRES award.
Congratulations! How did you achieve this?
I have won an INTERNATIONAL competition
I have published in an INTERNATIONAL journal
----------------------------------------
```

Experiment 3 Questions

1. In the interface `IBerprestasi`, change the `public` modifier of the *method* `menjuaraiKompetisi()` to `protected`, and that of the *method* `membuatPublikasiIlmiah()` to `private`. What happens? Explain why (include a screenshot of the result), then restore the code.
2. Add the line `IBerprestasi prestasi = new IBerprestasi();` to the `main()` method. Why does this line cause an error?
3. If an **outstanding Sarjana** must **win a NATIONAL competition** and **publish in a NATIONAL journal**, modify the related classes so that in the class `Program` the object `pakRektor` can present the Mawapres award to the object `sarjanaCumlaude`, with the following output.

Expected output:

```
I am the RECTOR, granting a MAWAPRES award.
Congratulations! How did you achieve this?
I have won a NATIONAL competition
I have published in a NATIONAL journal
---------------------------------------
I am the RECTOR, granting a MAWAPRES award.
Congratulations! How did you achieve this?
I have won an INTERNATIONAL competition
I have published in an INTERNATIONAL journal
---------------------------------------
```

IV. Assignment (± 65 minutes)

The UML diagram below describes a system that identifies the characteristics and food types of several animals (*Binatang*). A **Singa** (lion) only eats meat, so it is classified as a **Carnivore** (`IKarnivora`), while a **Keledai** (donkey) only eats plants, so it is a **Herbivore** (`IHerbivora`). A **Gorilla** can eat both kinds of food (an **omnivore**), so it is categorised as a **combination of Herbivore and Carnivore**. Every animal also has the characteristics **name, sound, fur colour, and number of legs**.

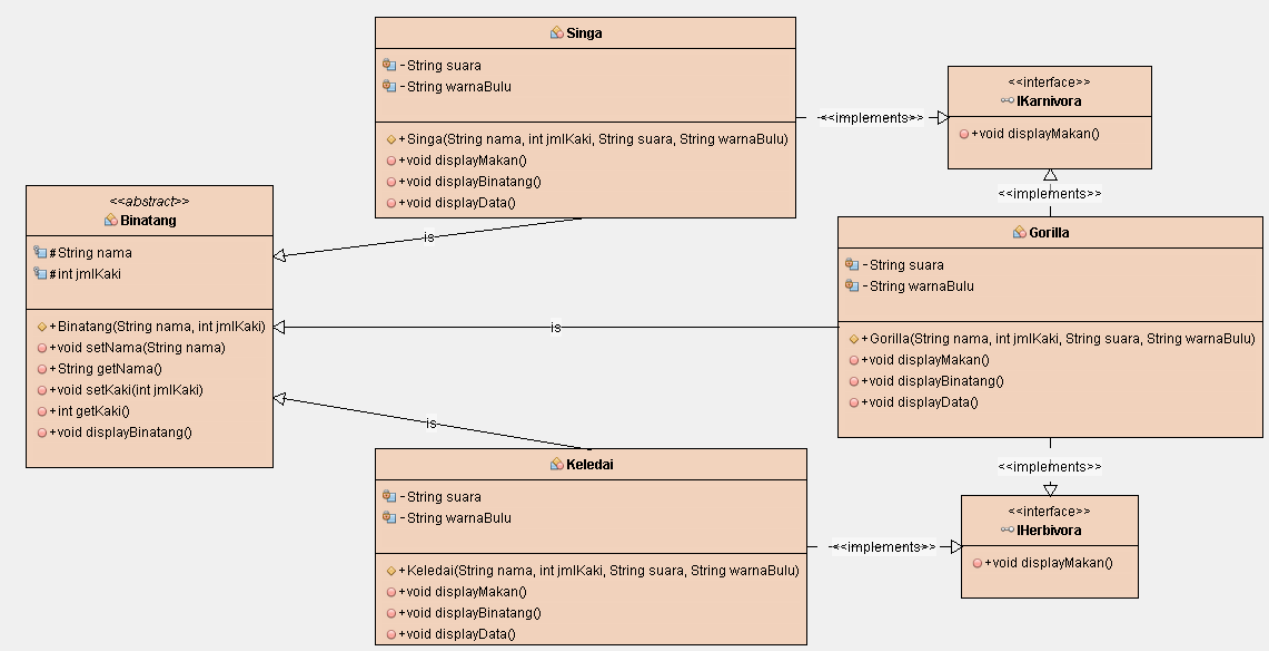


Figure 6. Class diagram for the assignment

Based on the case description and the UML diagram above, implement the system in Java (create a new package `tugas`) so that it produces output like the following.

Expected output:

```
Type: Herbivore
Food: Plants
Name: Keledai
Number of Legs: 4
Sound: Hehehehe
Fur Colour: Grey

Type: Carnivore + Herbivore
Food: Meat + Plants
Name: Gulali
Number of Legs: 4
Sound: Haaum Hauum
Fur Colour: Black

Type: Carnivore
Food: Meat
Name: CingaCing
Number of Legs: 4
Sound: Roaar Roaaar
Fur Colour: Brown
```

Hint: `Binatang` is an *abstract class* (an "is-a" relationship), while `IKarnivora` and `IHerbivora` are interfaces (a "can-do" relationship). The class `Gorilla` *implements* both interfaces at once. Because both interfaces declare the *method* `displayMakan()` with the same *signature*, `Gorilla` only needs to write **one** `displayMakan()` *method*.

What to submit: a report containing screenshots of your program code with explanations, screenshots of the program output, answers to all experiment questions, and the UML diagram of your system.

========== Good Luck ==========